МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2

Выполнил:
Студент группы ИБМ3-34Б

Фролова М.П.

Преподаватель:
Правдина А.Д.

Бурмистрова М.В.

Формулировка задания:

Написать программу, в которой описана иерархия классов: Планер, Самолет, Ракета. Базовый класс должен иметь методы получения потребления топлива для данного расстояния и вычисления времени движения на заданное расстояние. Базовый класс должен иметь метод вывода на экран характеристик экземпляра.

Виртуальное окружение и зависимости:

PS C:\Users\maria\Desktop\uni\git> python -m venv .venv

PS C:\Users\maria\Desktop\uni\git> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

PS C:\Users\maria\Desktop\uni\git> .venv\Scripts\Activate.ps1

(.venv) PS C:\Users\maria\Desktop\uni\git> pip install colorama

Collecting colorama

  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)

Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)

Installing collected packages: colorama

Successfully installed colorama-0.4.6

[notice] A new release of pip is available: 26.2.1 -> 26.3

[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) PS C:\Users\maria\Desktop\uni\git> pip show colorama

Name: colorama

Version: 0.4.6

Summary: Cross-platform colored terminal text.

Home-page: https://github.com/tartley/colorama

Author:

Author-email: Jonathan Hartley <tartley@tartley.com>

License:

Location: C:\Users\maria\Desktop\uni\git\.venv\Lib\site-packages

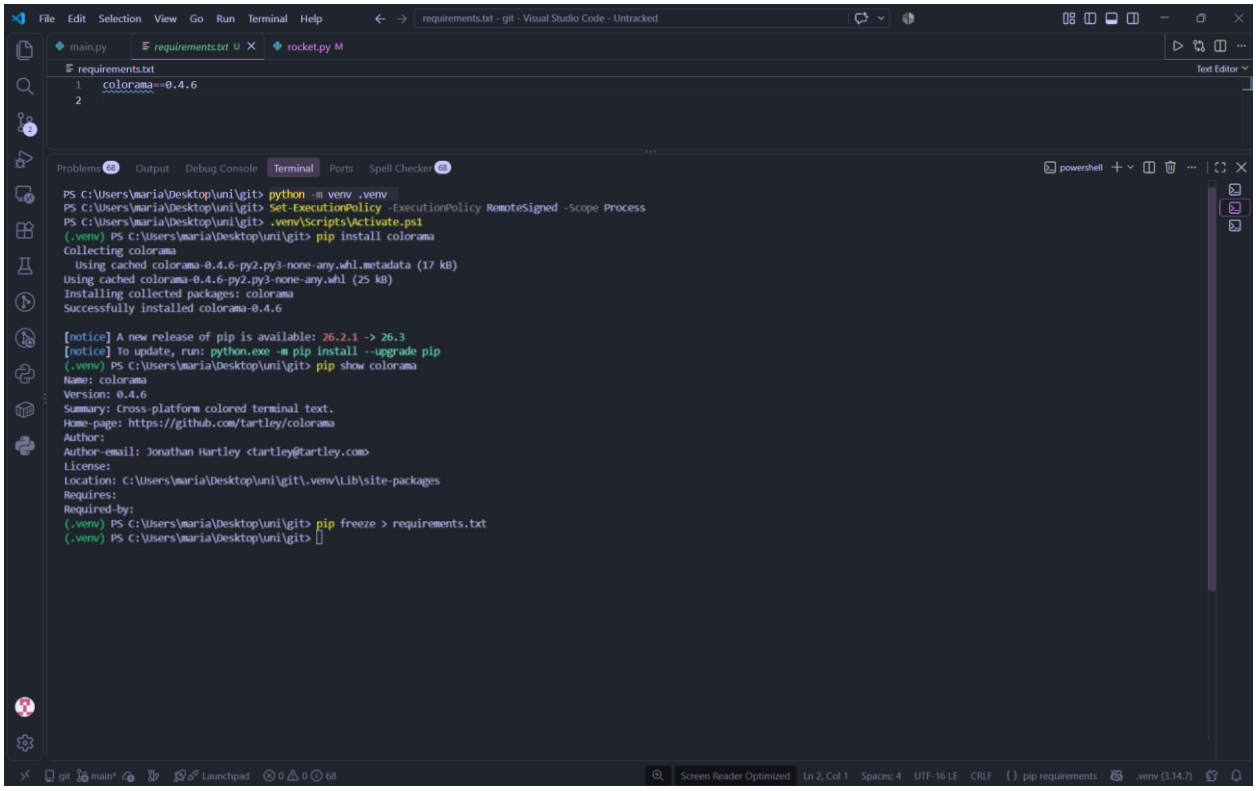
Requires:

Required-by:

(.venv) PS C:\Users\maria\Desktop\uni\git> pip freeze > requirements.txt

Содержимое файла requirements.txt:

colorama==0.4.6



Листинг программы:

Файл main.py:

```
from colorama import Fore, Style, init
from lab_python_oop.airplane import Airplane
from lab_python_oop.glider import Glider
from lab_python_oop.rocket import Rocket

def main():
  while True:
    print("\n======== МЕНЮ ========")
    print("1 - Запуск программы;")
    print("0 - Выйти из программы.\n")
    choice1 = input("Введите число (1, 0): ")
```

```
    if choice1 == "0":
      print("Выход.....")
      break

    if choice1 == "1":
      print("=== СИСТЕМА УПРАВЛЕНИЯ ЛЕТАТЕЛЬНЫМИ АППАРАТАМИ ===\n")

      print("Выберите тип объекта для демонстрации:")
      print("1. Планер")
      print("2. Самолет")
      print("3. Ракета")

      choice = input("Введите номер пункта (1-3): ").strip()

      obj = None
      if choice == "1":
        obj = Glider("Спортивный планер «АС-4»", 150.0)
      elif choice == "2":
        obj = Airplane("Пассажирский Airbus A320", 820.0, 3.8)
      elif choice == "3":
        obj = Rocket("Ракета-носитель «Falcon 1»", 14000.0, 85.0)
      else:
        print("Неверный выбор.")

      print("\n--- Демонстрация вывода через print (метод __str__) ---")
      print(obj)

      print("\n--- Проверка свойств (геттеры и сеттеры) ---")
      print(f"Текущая скорость: {obj.speed} км/ч")

      distance = int(input("Введите дистанцию: "))
      print(f"\n--- Расчеты для расстояния {distance} км ---")
      print(f"Время в пути: {obj.calculate_time(distance):.2f} ч")
      print(f"Требуемое топливо: {obj.calculate_fuel(distance)}")
      print(f"Статус полета: {obj.get_flight_status()}")
      continue
    else:
      print("Неверный ввод. Попробуйте еще раз.")

if __name__ == "__main__":
  main()
```

Файл fluying_object.py (базовый класс):

```
from abc import ABC, abstractmethod

class FlyingObject(ABC):
    def __init__ (self, name: str, speed: float):
        self.name = name
```

```
        self.speed = float(speed)

    @property
    def name(self) -> str:
        return self.__name

    @name.setter
    def name(self, value: str):
        if value =="":
            return ("Имя должно содержать в себе хотя бы какие-то символы")
        self.__name = value

    @property
    def speed(self) -> float:
        return self.__speed

    @speed.setter
    def speed(self, value: float):
        if value <= 0:
            return "Летательный объект не летит. Скорость должна быть больше 0."
        self.__speed = float(value)

    def calculate_time(self, distance: float) -> float:
        if distance <= 0:
            return "Расстояние не может быть отрицательным."
        return distance/self.__speed

    @abstractmethod
    def calculate_fuel(self, distance: float) -> float:
        pass

    def get_flight_status(self) -> str:
        return "аппарат выполняет полет в штатном режиме."

    def __str__(self) -> str:
        return (f"Летательный аппарат: {self.__name}, Скорость: {self.__speed}
км/ч")
```

Файл glider.py:

```
from lab_python_oop.flying_object import FlyingObject

class Glider(FlyingObject):
    def __init__(self, name = "Планер", speed = 100.0):
        super().__init__(name, speed)

    def calculate_fuel(self, distance: float) -> float:
        if distance < 0:
            return "Расстояние не может быть отрицательным"
```

```
        return 0.0

    def calculate_time(self, distance: float) -> float:
        return super().calculate_time(distance)
```

Файл airplane.py:

```
from lab_python_oop.flying_object import FlyingObject

class Airplane(FlyingObject):
    def __init__(self, name = "Самолет", speed = 900.0, fuel_consuption = 3.5):
        super().__init__(name, speed)
        self.__fuel_consuption = float(fuel_consuption)

    @property
    def fuel_consuption(self) -> float:
        return self.__fuel_consuption

    @fuel_consuption.setter
    def fuel_consuption(self, value: float):
        if value <= 0:
            return "Расход топлива должен быть положительным."
        self.__fuel_consuption = float(value)

    def calculate_time(self, distance: float) -> float:
        if distance <= 0:
            return "Расстояние не может быть отрицательным."
        base_time = super().calculate_time(distance)
        return base_time + 0.2

    def calculate_fuel(self, distance: float) -> float:
        if distance < 0:
            return "Расстояние не может быть отрицательным."
        return distance * self.__fuel_consuption

    def get_flight_status(self) -> str:
        return "Самолет следует заданному маршруту."

    def __str__(self) -> str:
        return(f"Самолет: {self.name}, Скорость: {self.speed} км/ч, Расход
топлива: {self.fuel_consuption} л/км")
```

Файл rocket.py:

```
from lab_python_oop.flying_object import FlyingObject

class Rocket(FlyingObject):
```

```
  def __init__(self, name: str = "Космическая ракета", speed: float = 11000.0,
fuel_consumption: float = 60.0,):
    super().__init__(name, speed)
    self.__fuel_consumption = float(fuel_consumption)

  @property
  def fuel_consumption(self) -> float:
    return self.__fuel_consumption

  @fuel_consumption.setter
  def fuel_consumption(self, value: float):
    if value <= 0:
      raise ValueError("Расход топлива ракеты должен быть положительным.")
    self.__fuel_consumption = float(value)

  def calculate_fuel(self, distance: float) -> float:
    if distance < 0:
      return "Расстояние не может быть отрицательным."
    return distance * self.__fuel_consumption

  def __str__(self) -> str:
    return (f"Ракета: «{self.name}», Скорость: {self.speed} км/ч, Расход
топлива:{self.__fuel_consumption} кг/км")
```

Вывод программы:

```
======== МЕНЮ ========
1 - Запуск программы;
0 - Выйти из программы.

Введите число (1, 0): 1
=== СИСТЕМА УПРАВЛЕНИЯ ЛЕТАТЕЛЬНЫМИ АППАРАТАМИ ===

Выберите тип объекта для демонстрации:
1. Планер
2. Самолет
3. Ракета
Введите номер пункта (1-3): 1

--- Демонстрация вывода через print (метод __str__) ---
Летательный аппарат: Спортивный планер «АС-4», Скорость: 150.0 км/ч

--- Проверка свойств (геттеры и сеттеры) ---
Текущая скорость: 150.0 км/ч
Введите дистанцию: 15000
```

```
--- Расчеты для расстояния 15000 км ---
Время в пути: 100.00 ч
Требуемое топливо: 0.0
Статус полета: аппарат выполняет полет в штатном режиме.

======== МЕНЮ ========
1 - Запуск программы;
0 - Выйти из программы.

Введите число (1, 0): 1
=== СИСТЕМА УПРАВЛЕНИЯ ЛЕТАТЕЛЬНЫМИ АППАРАТАМИ ===

Выберите тип объекта для демонстрации:
1. Планер
2. Самолет
3. Ракета
Введите номер пункта (1-3): 2

--- Демонстрация вывода через print (метод __str__) ---
Самолет: Пассажирский Airbus A320, Скорость: 820.0 км/ч, Расход топлива: 3.8 л/км

--- Проверка свойств (геттеры и сеттеры) ---
Текущая скорость: 820.0 км/ч
Введите дистанцию: 8200

--- Расчеты для расстояния 8200 км ---
Время в пути: 10.20 ч
Требуемое топливо: 31160.0
Статус полета: Самолет следует заданному маршруту.

======== МЕНЮ ========
1 - Запуск программы;
0 - Выйти из программы.

Введите число (1, 0): 1
=== СИСТЕМА УПРАВЛЕНИЯ ЛЕТАТЕЛЬНЫМИ АППАРАТАМИ ===

Выберите тип объекта для демонстрации:
1. Планер
2. Самолет
3. Ракета
Введите номер пункта (1-3): 3

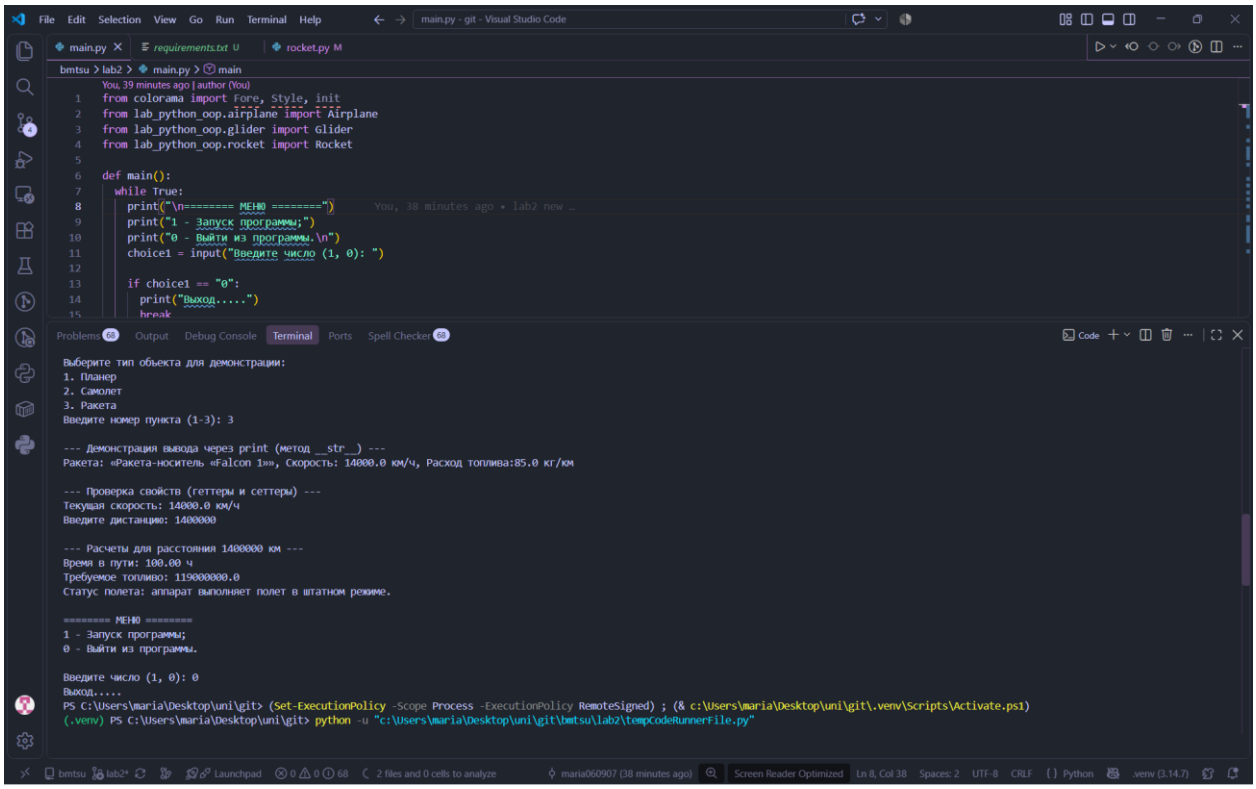
--- Демонстрация вывода через print (метод __str__) ---
Ракета: «Ракета-носитель «Falcon 1»», Скорость: 14000.0 км/ч, Расход топлива:85.0
кг/км

--- Проверка свойств (геттеры и сеттеры) ---
Текущая скорость: 14000.0 км/ч
Введите дистанцию: 1400000
```

```
--- Расчеты для расстояния 1400000 км ---
Время в пути: 100.00 ч
Требуемое топливо: 119000000.0
Статус полета: аппарат выполняет полет в штатном режиме.

======== МЕНЮ ========
1 - Запуск программы;
0 - Выйти из программы.

Введите число (1, 0): 0
Выход.....
```



Модульные тесты (файл testik_.py):

```
import unittest
from lab_python_oop.glider import Glider
from lab_python_oop.airplane import Airplane
from lab_python_oop.rocket import Rocket

class TestFlyingObjects(unittest.TestCase):

    def test_glider(self):
        glider = Glider("Спортивный планер «АС-4»", 150.0)
        self.assertEqual(glider.name, "Спортивный планер «АС-4»")
        self.assertEqual(glider.speed, 150.0)
        self.assertEqual(glider.calculate_time(300.0), 2.0)
        self.assertEqual(glider.calculate_fuel(300.0), 0.0)
```

```
    def test_airplane(self):
        plane = Airplane("Пассажирский Airbus A320", 800.0, 4.0)
        self.assertEqual(plane.name, "Пассажирский Airbus A320")
        self.assertEqual(plane.speed, 800.0)
        self.assertEqual(plane.fuel_consuption, 4.0)
        self.assertAlmostEqual(plane.calculate_time(800.0), 1.2)
        self.assertEqual(plane.calculate_fuel(100.0), 400.0)

    def test_rocket(self):
        rocket = Rocket("Ракета-носитель «Falcon 1»", 12000.0, 50.0)
        self.assertEqual(rocket.name, "Ракета-носитель «Falcon 1»")
        self.assertEqual(rocket.speed, 12000.0)
        self.assertEqual(rocket.fuel_consumption, 50.0)
        self.assertEqual(rocket.calculate_fuel(200.0), 10000.0)

if __name__ == "__main__":
    unittest.main()
```

Вывод:

(.venv) PS C:\Users\maria\Desktop\uni\git\bmtsu\lab2> python testik_.py

...

Ran 3 tests in 0.001s

OK